

Optimisation SEO d'une SPA Vanilla JS

Dans une application monopage (Single Page Application - SPA), l'optimisation pour les moteurs de recherche (SEO) est essentielle car tout le contenu est généré dynamiquement par JavaScript. Les moteurs de recherche ont besoin d'informations claires pour bien indexer votre site.

De plus, les bonnes pratiques d'optimisation des performances contribuent également à un meilleur référencement, car les moteurs de recherche favorisent les sites rapides et performants. Le chargement rapide des pages améliore l'expérience utilisateur, ce qui est un facteur clé pour le SEO.

Nous verrons la minification et la compilation dans un cours ultérieur, mais pour l'instant, concentrons-nous sur les optimisations SEO de base et les bonnes pratiques de performance.

Pourquoi le SEO est important pour une SPA ?

Le défi : Dans une SPA, le contenu change sans recharger la page. Les moteurs de recherche (Google, Bing, etc.) doivent pouvoir :

- Comprendre le contenu de chaque "page" virtuelle
- Indexer les différentes sections de votre application
- Afficher des aperçus corrects dans les résultats de recherche

1. Gestion dynamique des balises <head>

Quand l'utilisateur navigue dans votre SPA, vous devez mettre à jour les balises meta pour refléter le contenu actuel.

Voici les balises principales à gérer :

- **<title>**: Le titre de la page, visible dans l'onglet du navigateur et les résultats de recherche.
- **<meta name="description">**: La description de la page, utilisée par les moteurs de recherche pour les aperçus.
- **<link rel="canonical">**: L'URL canonique de la page, pour éviter les problèmes de contenu dupliqué. Chaque URL doit pointer vers une version unique de la page.
- Open Graph tags (**<meta property="og:title">**, **<meta property="og:description">**, etc.) : Pour optimiser le partage sur les réseaux sociaux. Cela permet de contrôler l'apparence des liens partagés sur des plateformes comme Facebook, X, LinkedIn, etc.

Balises essentielles à mettre à jour

```
<!-- Dans votre index.html de base -->
<head>
  <title>Titre par défaut</title>
  <meta name="description" content="Description par défaut" />
  <link rel="canonical" href="https://votresite.com/" />

  <!-- Open Graph (pour Facebook, LinkedIn) -->
  <meta property="og:title" content="Titre par défaut" />
  <meta property="og:description" content="Description par défaut" />
```

```
<meta property="og:image" content="https://votresite.com/image.jpg" />
<meta property="og:url" content="https://votresite.com/" />
</head>
```

Module pour gérer le <head> dynamiquement

Créez un fichier `head-manager.js` :

```
// head-manager.js
class GestionEntete {
  /**
   * Change le titre de la page
   */
  static setTitle(title) {
    document.title = title;
  },

  /**
   * Met à jour une balise meta par son attribut "name"
   */
  static setMeta(name, content) {
    let metaTag = document.querySelector(`meta[name="${name}"]`);

    if (!metaTag) {
      // Créer la balise si elle n'existe pas
      metaTag = document.createElement("meta");
      metaTag.setAttribute("name", name);
      document.head.appendChild(metaTag);
    }

    metaTag.setAttribute("content", content);
  },

  /**
   * Met à jour une balise Open Graph (property)
   */
  static setOpenGraph(property, content) {
    let ogTag = document.querySelector(`meta[property="${property}"]`);

    if (!ogTag) {
      ogTag = document.createElement("meta");
      ogTag.setAttribute("property", property);
      document.head.appendChild(ogTag);
    }

    ogTag.setAttribute("content", content);
  },

  /**
   * Met à jour l'URL canonique
   */
  static setCanonical(url) {
```

```

let canonicalLink = document.querySelector('link[rel="canonical"]');

if (!canonicalLink) {
  canonicalLink = document.createElement("link");
  canonicalLink.setAttribute("rel", "canonical");
  document.head.appendChild(canonicalLink);
}

canonicalLink.setAttribute("href", url);
},

/**
 * Met à jour toutes les balises pour une page
 */
static updatePage(pageData) {
  GestionEntete.setTitle(pageData.title);
  GestionEntete.setMeta("description", pageData.description);
  GestionEntete.setCanonical(pageData.url);

  // Open Graph
  GestionEntete.setOpenGraph("og:title", pageData.title);
  GestionEntete.setOpenGraph("og:description", pageData.description);
  GestionEntete.setOpenGraph("og:url", pageData.url);

  if (pageData.image) {
    GestionEntete.setOpenGraph("og:image", pageData.image);
  }
},
};

export default GestionEntete;

```

Utilisation dans votre routeur

Dans chacune des vues, vous pouvez appeler `GestionEntete.updatePage()` avec les données spécifiques à la page :

```

import GestionEntete from "../GestionEntete.js";

// Exemple pour la page "Produits"
function render() {
  GestionEntete.updatePage({
    title: "Nos Produits - Mon Site",
    description: "Découvrez notre gamme complète de produits.",
    url: "https://monsite.com/produits",
    image: "https://monsite.com/images/produits.jpg",
  });

  // Rendu du contenu de la page...
}

```

2. Le fichier `sitemap.xml`

Le sitemap est un fichier XML qui liste toutes les pages de votre site pour aider les moteurs de recherche à les découvrir. Pour chaque URL de votre SPA, ajoutez une entrée dans le sitemap avec la fréquence de mise à jour et la priorité.

Il existe des outils en ligne pour générer automatiquement un sitemap, mais voici un exemple simple de structure.

Structure d'un `sitemap.xml`

Créez un fichier `sitemap.xml` à la racine de votre site :

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://monsite.com/</loc>
    <changefreq>weekly</changefreq>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://monsite.com/produits</loc>
    <changefreq>weekly</changefreq>
    <priority>0.8</priority>
  </url>
  <url>
    <loc>https://monsite.com/a-propos</loc>
    <changefreq>monthly</changefreq>
    <priority>0.5</priority>
  </url>
  <url>
    <loc>https://monsite.com/contact</loc>
    <changefreq>monthly</changefreq>
    <priority>0.5</priority>
  </url>
</urlset>
```

Explication des balises

- `<loc>` : L'URL complète de la page
- `<changefreq>` : Fréquence de mise à jour (always, hourly, daily, weekly, monthly, yearly, never)
- `<priority>` : Importance relative de la page (0.0 à 1.0)

Fichier `robots.txt`

Créez aussi un fichier `robots.txt` à la racine pour indiquer où se trouve votre sitemap :

```
User-agent: *
Allow: /
```

3. Lazy Loading des images

Le lazy loading (chargement différé) permet de charger les images et des vidéos uniquement quand elles sont sur le point d'être visibles. Cela améliore considérablement les performances initiales.

Attribut `loading="lazy"` (Natif)

La façon **la plus simple** et recommandée est d'utiliser l'attribut natif `loading="lazy"` sur les balises `<img>`. Cela permet au navigateur de gérer le chargement différé automatiquement.

```

```

Images responsive avec lazy loading

Une autre solution est d'avoir des images ajustées en fonction de la taille de l'écran tout en utilisant le lazy loading. On indique plusieurs sources avec `srcset` et `sizes`. L'image la plus appropriée sera chargée en fonction de la taille de l'écran de l'utilisateur.

```

```

4. Bonnes pratiques SEO pour SPA

☒ À faire

1. **Toujours mettre à jour le titre et la description** à chaque changement de page
2. **Utiliser des URL significatives** : `/produits/chaussures` plutôt que `/page?id=123`
3. **Ajouter du contenu textuel** : Les moteurs de recherche ont besoin de texte
4. **Utiliser des balises sémantiques** : `<article>`, `<section>`, `<nav>`, `<header>`, etc.
5. **Optimiser les images** : format WebP, compression, dimensions appropriées
6. **Tester avec les outils Google** : Google Search Console, test de résultats enrichis

✗ À éviter

1. **Ne pas mettre tout le contenu dans un seul fichier JavaScript énorme**
2. **Ne pas oublier les attributs `alt` sur les images**
3. **Ne pas bloquer l'indexation dans robots.txt**

5. Vérifier votre SEO

Une fois votre site déployé, utilisez des outils pour vérifier que tout est en ordre. Google Search Console est un excellent point de départ. Cela permet de soumettre votre sitemap, vérifier l'indexation et détecter les erreurs.

L'outil Lighthouse dans Chrome DevTools est également très utile pour auditer les performances et le SEO de votre site.

Outils de test

1. **Google Search Console** : <https://search.google.com/search-console>
 - Soumettre votre sitemap
 - Vérifier l'indexation de vos pages
 - Voir les erreurs d'exploration
2. **Test d'optimisation mobile** : <https://search.google.com/test/mobile-friendly>
3. **Lighthouse** (dans Chrome DevTools)
 - Ouvrir DevTools (F12)
 - Onglet "Lighthouse"
 - Générer un rapport

Ressources utiles

- [Google - Guide du SEO](#)
- [MDN - Lazy Loading](#)
- [web.dev - Optimize LCP](#)